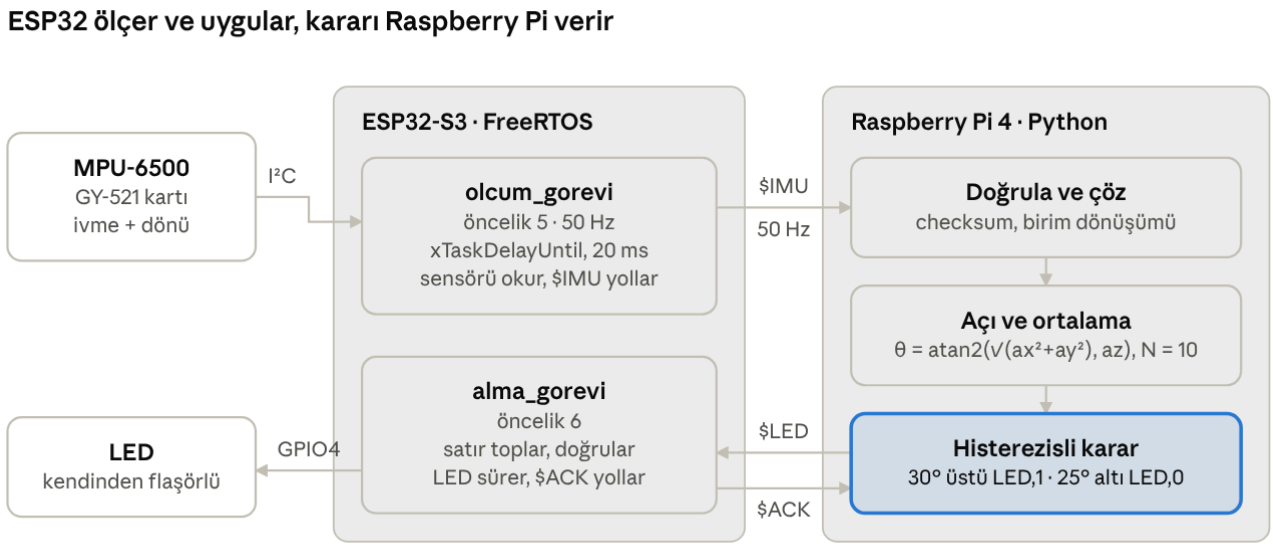
ESP32-S3 ile Raspberry Pi Haberleşmesi ve Eğim Alarmı

Ders: İleri Gömülü Sistemler (Yüksek Lisans) · **Vize Projesi Raporu** · 9 Kasım 2026
Öğrenci: Yasir YILDIRIM — 2610140003 · **Seçtiğim ekleme:** V1 — Eğim alarmı · **Depo:** `vize-2610140003`

1. Amaç

Bu projede ESP32-S3 ile ivme sensörünü 50 Hz'de okudum ve verileri UART üzerinden Raspberry Pi'ye gönderdim. Pi gelen satırları checksum ile kontrol ediyor, CSV'ye kaydediyor ve ESP32'ye komut gönderebiliyor. Ek olarak **V1 eğim alarmını** seçtim: Pi, ivmeden sensörün ne kadar eğildiğini hesaplıyor; açı 30°'yi geçince ESP32'ye `$LED,1`, 25°'nin altına inince `$LED,0` gönderiyor. Yani Pi sadece veri toplamıyor, veriye bakıp karar da veriyor.

2. Mimari



Zamanlaması önemli işleri ESP32'ye bıraktım. `olcum_gorevi` (öncelik 5) sensörü `xTaskDelayUntil` ile her 20 ms'de bir okuyup `$IMU` satırı gönderiyor. `alma_gorevi` (öncelik 6) ise UART'tan gelen komutları bekliyor; önceliği daha yüksek olduğu için komutlara gecikmeden cevap verebiliyor. Kararı Pi tarafında, Python ile veriyorum; her örneği ve her alarm olayını CSV dosyalarına yazıyorum.

3. Bağlantı

ESP32-S3	Bağlandığı yer	Görevi	ESP32-S3	Bağlandığı yer	Görevi
GPIO17 (UART1 TX)	Pi pin 10 (RXD)	ESP32 → Pi veri	3V3	Sensör VCC	Sensör beslemesi
GPIO18 (UART1 RX)	Pi pin 8 (TXD)	Pi → ESP32 komut	GPIO8 / GPIO9	Sensör SDA / SCL	I²C, 400 kHz
GND	Breadboard GND hattı (Pi pin 6 ve sensör GND de burada)	Ortak toprak	GPIO4	330 Ω → LED → GND	Alarm göstergesi

Kartları ayrı USB kablolarından besledim, Pi'nin 5V pinlerine bir şey bağlamadım. Pi'de seri konsolu kapatıp UART'ı açtım (`raspi-config`). LED'e 330 Ω direnç koydum: 3,3 V'tan LED'in ~2 V'u düşünce kalan 1,3 V dirençte düşüyor ve akım ~4 mA ile sınırlanıyor.

4. Protokol

Her satır `$GOVDE*HH\r\n` biçiminde. HH, `$` ile `*` arasındaki karakterlerin XOR'u (örnek: `$LED,1*50`). Hız 115200 baud, 8N1.

Yön	Mesaj	Açıklama
ESP32 → Pi	`$IMU,t,ax,ay,az,gx,gy,gz*HH`	t ms; ivme 16384 LSB/g, dönü 131 LSB/(°/s)
ESP32 → Pi	`$ACK,LED,0\|1` · `$ACK,HIZ,1..100`	Komut uygulandı
ESP32 → Pi	`$ERR,BICIM|CHECKSUM|KOMUT|I2C`	Hatalı satır, bilinmeyen komut, sensör okunamadı
Pi → ESP32	`$LED,0\|1*HH` · `$HIZ,1..100*HH`	LED; örnekleme hızı (Hz)

Eklemem protokole yeni bir mesaj eklemiyor. Var olan `$LED` komutunu kullanıyorum ama sadece alarm durumu değiştiğinde gönderiyorum. Gelen her `$ACK`'in ne kadar sürede geldiğini de olay dosyasına yazıyorum.

5. Eklemem: eğim alarmı (`pi/egim_alarm.py`)

Program her IMU satırında şu adımları uyguluyor (ESP32 kodunu değiştirmedim):

1. **Açıyı hesaplıyorum.** Sensör dururken ölçtüğü tek ivme yerçekimi. Sensörün Z ekseni ile yerçekimi arasındaki açıyı eğim olarak aldım: düz 0°, dik 90°, ters 180°. Bu açı sensör hangi yöne eğilirse eğilsin doğru çıkıyor.
2. **Ortalama alıyorum.** Gürültü yüzünden alarm titremesin diye son 10 örneğin (0,2 s) ortalamasını kullanıyorum. Açıların değil, ivme bileşenlerinin ortalamasını alıp açıyı ondan hesaplıyorum (nedenini Bölüm 7'de anlattım).
3. **Histerezis kullanıyorum.** Alarm 30°'de açılıyor, 25°'de kapanıyor. Aradaki 5°'lik bant sayesinde açı eşiğin çevresinde gidip gelirken LED sürekli yanıp sönmüyor.
4. **Komutu sadece durum değişince gönderiyorum.** Böylece hatta her satırda gereksiz komut gitmiyor. Program açılırken ve kapanırken LED'i söndürüyor.

$$\bar{a} = (1/N)\ \Sigma\ a_{k-i} \qquad \bar{\theta} = \text{atan2}(\ \sqrt{(\bar{a}_x^2 + \bar{a}_y^2)},\ \bar{a}_z\ ) \cdot 180/\pi$$

İlk sürümde yönergedeki θ = atan2(ax, az) formülünü kullandım; bu formül `--tek-eksen` seçeneğiyle hâlâ duruyor. Ama denerken bu formülün sadece bir eksendeki eğimi ölçtüğünü gördüm: sensörü diğer yöne eğince açı hiç değişmiyordu. Bu yüzden varsayılanı yukarıdaki formülle değiştirdim. Eşikleri ve pencereyi komut satırından ayarlayabiliyorum (`--ust 30 --alt 25 --pencere 10`).

6. Ölçüm

Özetle: sistem 50 Hz'de kayıpsız çalıştı, komutlar ~3,4 ms'de cevap aldı ve histerezis LED'in titremesini yaklaşık 6 kat azalttı. Gerçek sensörle aldığım dört kayıtta toplam 10.153 satırdan sadece port açılırken yarım okunan ilk satır hatalı çıktı.

6.1 Örnek hızı ve hatalı satır

Kayıt	Süre	IMU satırı	Hatalı	Sonuç
60 sn kayıt, gerçek sensör (`kayit_60s.csv`)	60,4 s	3.021	0	50,0 örnek/s; aralık 20,0 ± 0,58 ms (19–21)
Diğer üç kayıt, gerçek sensör	142,5 s	7.132	1 (ilk yarım satır)	50,0 örnek/s
Uzun çalışma, sensör bağlı değilken (sentetik veri)	~3,4 dk	10.230	0	50,0 örnek/s

Aralıktaki ±1 ms'lik oynama, ESP32'nin zaman damgasını milisaniyeye yuvarlamasından geliyor; periyot kaymadı.

6.2 `hiz` komutu ve komut → ACK süresi

Komut	Yanıt	Beklenen	Ölçtüğüm periyot (ort. ± ss)	n
`hiz 10`	`ACK,HIZ,10`	100 ms	100,0 ± 0,8 ms	152
`hiz 100`	`ACK,HIZ,100`	10 ms	10,0 ± 0,0 ms	1.230
`hiz 50`	`ACK,HIZ,50`	20 ms	20,0 ± 0,0 ms	605

Periyodu ESP32'nin zaman damgalarından hesapladım. `alici.py` ekranındaki "örnek/s" değeri programın başından beri ortalama olduğu için yeni hızı geç gösterdi (`hiz 10`'dan sonra 41,9 → 31,8 → … → 19,6 diye yavaş yavaş indi).

`ack_olc.py`, 30 tekrar	Ortalama	Medyan	Std. sapma	Min	p95	Maks	Kayıp
`$LED` → `$ACK`	3,88 ms	3,38 ms	0,97 ms	3,36 ms	5,92 ms	5,96 ms	0 / 30

Sürelerin çoğu ~3,4 ms. Komut (11 bayt) ve ACK (15 bayt) 115200 baud'da toplam ~2,3 ms sürüyor, gerisi işlem süresi. Her 5–6 tekrarda bir süre ~5,9 ms'ye çıkıyordu. Bunun nedenini şöyle buldum: ESP32 aynı UART'tan her 20 ms'de ~44 baytlık (~3,8 ms) bir IMU satırı gönderiyor ve ACK, o sırada gitmekte olan satırın bitmesini bekliyor. Tekrarlar ~203,4 ms arayla geldiği için komut her seferinde IMU döngüsüne göre 3,4 ms kayıyor ve yaklaşık 20/3,4 ≈ 6 tekrarda bir IMU satırına denk geliyor. Eğim alarmı çalışırken gönderilen 140'tan fazla `$LED` komutunun da hepsi 3,3–3,4 ms'de onaylandı.

6.3 Açı doğruluğu ve histerezis

Konum	Gerçek	Ölçülen $\bar{\theta}$	Hata		Histerezis deneyi (eşik çevresinde aynı el hareketi)	Süre	LED değişimi
Masada düz	0°	1,2°	+1,2°		Histerezis ve ortalama yok (`--ust 30 --alt 30 --pencere 1`)	32,8 s	**112** (3,4/s)
Kitaba dik yaslı	90°	90,3° (88,7–91,3)	+0,3°		Histerezis + 10 örnek ortalama (varsayılan)	35,5 s	**20** (0,56/s)

Alarm 30,3°'de açıldı, 24,1°'de kapandı. Açıölçerim olmadığı için ara açıları ölçemedim, sadece düz ve dik konumu kullandım. Sensör dururken toplam ivme 1 g yerine 1,06–1,07 g okunuyor; sensörün Z ekseninde ~%6–7 hata var ama açıyı oranla hesapladığım için sonuca etkisi az oldu. Histerezis olmadan 30,2° → 29,9° gibi çok küçük oynamalar bile LED'i değiştirdi. Bir sınırlılık da şu: sensörü hızlı sallayınca hareketin kendi ivmesi de ölçüldüğü için açı bir anlığına sıçradı; ortalama bunu azalttı ama tamamen gidermedi.

7. Karşılaştığım sorunlar

Ne oldu	Nasıl anladım	Nasıl çözdüm
Monitörde `probe device timeout` ve "Sensör yok" yazdı; sensör kartının LED'i yanmıyordu	Hata "cevap yok (NACK)" değil "zaman aşımı"ydı; I²C çekme dirençleri sensör kartının üzerinde olduğu için kart beslenmeyince hatlar boşta kalıyor. Kartı bükünce LED yanıyordu, yani pinler lehimle bağlanmamıştı	VCC, GND, SCL ve SDA pinlerini yeniden lehimledim
Saniyede 50 yerine ~263 satır geldi; ESP32 zaman damgası satır başına ~3,8 ms artıyordu	Sensör teması gidip gelirken her başarısız okuma 100 ms zaman aşımı beklemiş; periyot 20 ms olduğu için görev her seferinde 80 ms geride kalmış. Temas düzelince `xTaskDelayUntil` hiç beklemeden birikmiş örnekleri UART'ın en yüksek hızında (115200 ÷ 10 ÷ ~44 ≈ 262 satır/s) göndermiş	RST ile sıfırladım ve teması düzelttim. `vTaskDelay` kullanılsaydı bu patlama olmazdı ama periyot okuma süresi kadar kayardı
Veri düzenli bir sinüs gibiydi, az hep 16384, jiroskop hep 0	Firmware açılışta sensörü bulamayınca sahte (sentetik) veri gönderiyor; GND kablosunu breadboard'a taşırken temas kopmuştu	Bağlantıyı düzeltip RST'ye bastım
`WHO_AM_I = 0x70` okundu (beklenen 0x68)	0x70, MPU-6500'ün kimlik değeri. Kullandığım registerlar (0x1A–0x1C, 0x3B–0x48, 0x6B) ve ölçekler iki yongada aynı	Firmware'i değiştirmem gerekmedi
Sensörü ters çevirince iki kez yanlış alarm oldu ($\bar{\theta}$ = +30,4°, +35,6°)	İlk sürüm açıların ortalamasını alıyordu; +179° ile −174° neredeyse aynı yön ama düz ortalamaları anlamsız bir değer veriyor	İvme bileşenlerinin ortalamasına geçtim; tekrar denediğimde yanlış alarm olmadı

Ne oldu	Nasıl anladım	Nasıl çözdüm
Sensörü elle eğince açı bazen hiç değişmedi ya da eksiye düştü	atan2(ax, az) sadece tek eksendeki eğimi ölçüyor	atan2(√(ax²+ay²), az) formülüne geçtim
`idf.py set-target` hata verdi, yolda `gömülü` → `g├Âm├╝l├╝` oldu	ESP-IDF, Türkçe karakter ve boşluk içeren yolu bozuyor; `subst` ile kısayol da işe yaramadı	Projeyi `C:\esp32_vize`'ye kopyalayıp oradan derledim
Yüklerken “No serial data received”, sonra COM3 kayboldu	Kartın kendi USB girişinde otomatik yükleme moduna geçiş olmadı; elle geçince COM numarası 3'ten 4'e değişti	BOOT + RST ile yükleme moduna alıp COM4'ten yükledim

8. Sonuç

ESP32 ile Pi arasındaki haberleşme 50 Hz'de hatasız çalıştı ve komutlar (`$LED` → `$ACK`) medyan 3,38 ms'de cevap aldı. Eğim alarmıyla Pi veriye bakıp karar veriyor ve ESP32'deki LED'i sürüyor; histerezis titremeyi 112 değişimden 20'ye indirdi. Bu projede en çok şunu öğrendim: gevşek bir lehim gibi basit bir donanım sorunu, yazılımda hiç beklemediğim bir zamanlama hatası olarak ortaya çıkabiliyor.

Yapay zekâ kullanım beyanı. `egim_alarm.py` ve `ack_olc.py` betiklerini, README'yi ve bu raporun taslağını Claude (Anthropic) yapay zekâ asistanının yardımıyla hazırladım; kurulum ve hata ayıklama sırasında da ondan yardım aldım. Donanımı kurmak, lehimlemek, kodu çalıştırmak ve bütün ölçümleri almak bana ait. Kullandığım kodu satır satır inceledim ve açıklayabilirim. ESP32 firmware'i ile `protokol.py` ve `alici.py` dersin şablonundan değiştirilmeden alındı.